# Software Requirements Specification

**for**

# Calowin

**Version 1.0 approved**

**Prepared by**

RAPHEL SIM YANG YUE,
CHEN ZIYAN,
GUAN XIAOXI,
LIM JUN JIE,
TAN YI DA

**Nanyang Technological University**

**27 August 2024**

# Table of Contents

# Revision History

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| | | | |
| | | | |

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification document specifies the requirements for the Calowin application, including the scope of features, functionalities, and constraints. The document focuses on version 1.0 of the product, which aims to promote healthier lifestyles while reducing users' carbon footprint, by offering tools for users to find wellness zones, track calories burned and carbon saved during trips, check the rank among friends, and more. The SRS covers the full system and its integration with various user activities.

## 1.2 Document Conventions

This section describes the conventional standards were followed when writing this SRS document.

**Font:** Times New Roman
**Heading:** Bold, Size 18
**Sub-heading:** Bold, Size 14
**Content:** Italic, Size 12
**Technical Standards:** IEEE-830-1998

## 1.3 Intended Audience and Reading Suggestions

This document is intended for all stakeholders, including the users of the Calowin Mobile Application, the Calowin development team, and the Calowin business analyst and project manager team.

**All stakeholders** of the Calowin Application are recommended to begin by reviewing Appendix A: Data Dictionary, which provides a comprehensive list of terms and definitions essential for understanding the project's context. Familiarizing themselves with these terms will help ensure a shared understanding across all parties involved.

**The development team** should start by reading Section 2: Overall Description, followed by Section 3: External Interface Requirements and Section 4: System Features as it offers a high-level overview of the system, including the problem statement, goals, and general design.

This section serves as the foundation for understanding the rest of the document. By following this reading path, both stakeholders and the development team will gain a solid understanding of the terminology, objectives, and the overall scope of the project, which will enable them to engage effectively in the development process.

**All users** of the Calowin Mobile Application are encouraged to focus on Section 3.1: User Interfaces and Section 4: System Features to gain a clear understanding of the application's functionalities and how to navigate its features effectively. Additionally, reviewing Section 5: Other Non-Functional Requirements can provide insights into the app's performance, security, and usability standards, ensuring users are aware of the quality and reliability they can expect. By familiarizing themselves with these sections, users will be better equipped to utilize the application efficiently and provide informed feedback.

By following these tailored reading paths, all parties involved will achieve a comprehensive understanding of the project's terminology, objectives, and scope, facilitating effective collaboration and successful development of the Calowin Mobile Application.

## 1.4 Product Scope

Calowin is a mobile application designed to empower environmentally conscious individuals and fitness enthusiasts by enabling them to effectively track and manage their carbon footprints while monitoring their physical activity during trips.

## 1.5 References

*<List any other documents or Web addresses to which this SRS refers. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location.>*

a. Source Code: https://github.com/SC2006-AY24S1/SC2006-Software-Engineering/tree/main
b. NParks Parks and Nature Reserves (KML) https://data.gov.sg/datasets/d_04fc06188bf98f3c97d2c7ceeabee76f/view
c. 2-hour Weather Forecast https://data.gov.sg/datasets/d_3f9e064e25005b0e42969944ccaf2e7a/view

d. Google Maps API
https://developers.google.com/maps

# 2. Overall Description

## 2.1 Product Perspective

The *Calowin* mobile application is an extension of existing navigation applications, such as CityMapper. Unlike traditional navigation tools, Calowin does not provide route planning or navigation functionalities. Instead, it emphasizes environmental and fitness tracking during a trip, calculating metrics such as carbon saved and calories burnt, complemented by social engagement features to motivate and support users in their sustainability and health goals.

## 2.2 Product Functions

The Calowin mobile application's product function will be broken into 6 main features - Accounts, User Profile, Trips, Wellness Zone, Friends and Leaderboard. In depth analysis and specifications into each features can be found under Section 4: System Features

### 2.2.1 Accounts

2.2.1.1. Account Sign Up

2.2.1.2. Account Login

2.2.1.3. Forget Password

2.2.1.4. Restore Password

### 2.2.2 User Profile

2.2.2.1. View Profile (Self and Friends)

2.2.2.2. Edit Profile

2.2.2.3. Change Password

2.2.2.4. Delete Account

### 2.2.3 Trips

2.2.3.1 Start Trip

2.2.3.2. Cancel Trip

2.2.3.3. End Trip

2.2.3.4 Retrieve Trip Metrics

2.2.3.5. Map Functionalities and Routing

**2.2.4 Wellness Zones**

2.2.4.1. View Wellness Zone

2.2.4.2. Visit Wellness Zone

**2.2.5 Friends**

2.2.5.1. Add Friends

2.2.5.2. Reject Friend Requests

2.2.5.3. Send Friend Requests

2.2.5.4. View Friend Profile

**2.2.6 Leaderboard**

2.2.6.1. View Leaderboard

2.2.6.2. Filter Leaderboard

## 2.3 User Classes and Characteristics

The Calowin mobile application expects its target audience to include the following groups, listed in order of priority with characters:

2.3.1. **Fitness Enthusiasts**

2.3.1.1 Users who consistently track their fitness activities and engage with the app's social features. They are typically highly motivated by leaderboards and achievement badges. The *Calowin* marketing team emphasize that this group needs is essential for the app's success, as they are likely to provide valuable feedback, drive app usage, and influence other users.

2.3.2. **Health-Conscious Users**

2.3.2.1. Individuals aiming to improve their health through physical activity and social interaction, with diverse fitness goals. The *Calowin* marketing team views this group as an essential growth segment for the app, as they represent users with broad fitness goals who may be open to integrating more regular activity into their routines.

2.3.3. **Occasional Users**

2.3.3.1. Individuals who use the app intermittently, primarily for basic features such as locating wellness zones or tracking specific trips. The marketing team views this group as an untapped opportunity to increase user retention and engagement. While they currently use the app on a more casual basis, these users could be drawn to additional features like personalized notifications, occasional challenges, or rewards for consistent check-ins.

## 2.4 Operating Environment

**Hardware Platform**: Mobile devices with GPS and internet connectivity.

**Operating System:** Android or iOS, supporting the latest versions as well as previous major versions (Android 10 and above, iOS 13 and above).

| **Development Environment** |
|---|
| **Frontend**<br>Framework: Flutter<br>Programming Language: Dart |
| **Backend**<br>Environment: Virtual Machine<br>Programming Language: Java |

**Other Software Components:** Integration with Google Maps API (Places API, Maps SDK for Android, Directions API, Service Usage API, Maps SDK for iOS), SQLite for database management, OTP service for email verification.

## 2.5 Design and Implementation Constraints

2.5.1. **Regulatory Compliance**

2.5.1.1. The app must comply with data privacy regulations (PDPA for Singaporean users).

2.5.2. **Hardware Limitations**

2.5.2.1 The app must be optimized to run efficiently on devices with limited memory and processing power.

2.5.3. **API Integration**

2.5.3.1. The app is dependent on third-party APIs for multiple services. Any changes to these APIs could affect app functionality.

2.5.4. **Design Conventions**:

2.5.4.1. The app should follow standard mobile UI/UX design guidelines to ensure a consistent user experience across devices.

2.5.5. **Language**

2.5.5.1 The current version of the Calowin mobile application only supports the English language.

## 2.6 User Documentation

2.6.1. **User Manual**

2.6.1.1. A comprehensive guide that explains how to install the app and use the app's features including account setup and other basic functions in the app, provided in README.md file on the Github Repository which contains the source code (see **Section 1.5** References).

## 2.7 Assumptions and Dependencies

2.7.1. **Assumptions**

2.7.1.1. Users will have access to a stable internet connection and GPS services.

2.7.1.2. The third-party APIs will remain available and maintain backward compatibility during the app’s lifecycle.

2.7.2. **Dependencies**

2.7.2.1. **Third-Party APIs**

2.7.2.1.1. The app relies on APIs for location-based services. Any disruptions to this service could impact the app's core functionality.

2.7.2.2. **Device Compatibility**

2.7.2.2.1. The app's performance and availability depend on compatibility with Android 10 and above or iOS 13 and above. Updates to these operating systems may require adjustments to the app.

# 3. External Interface Requirements

## 3.1 User Interfaces

Please refer to UI mockups in the Github Repository for User Interface Mockups.

## 3.2 Hardware Interfaces

The *Calowin* mobile application requires a mobile phone that supports either the Android or iOS platform. It is assumed that the information obtained from these APIs are accurate. The device must have Internet connection capabilities to communicate with the database server and the APIs.

## 3.3 Software Interfaces

The Calowin mobile application utilizes a back-end server, implemented with Java 17 using the Spring framework, to manage communication with individual modules for each feature and to handle all system functionalities. Microsoft SQL Server is used as the relational database, providing persistent data storage in SQL format.

The application retrieves location data from the device's GPS to calculate distances and route metrics. Additionally, it integrates external datasets, including the NParks Parks and Nature

Reserves (KML) and the 2-hour Weather Forecast API from the Singapore Government website, to enhance location-based and weather-related features.

### 3.4 Communications Interfaces

The Calowin mobile application's communication architecture will follow a client-server model, with all interactions channeled through a REST-style API hosted by the back-end server. Built with Flutter for the client-side, the application ensures a seamless, cross-platform experience across mobile and web. All communications between the Flutter frontend and the server are secured over HTTPS.

Client-to-server interactions from Flutter will utilize GET and POST requests, while server responses will return data in standardized JSON format, allowing for easy parsing and display within Flutter's widget-based UI. For external integrations, the application will use JDBC (Java Database Connectivity) for communication with Microsoft SQL Server. Access to Google Maps services will be handled through RESTful API calls, while interactions with the Nparks and 2-hour Weather Forecast APIs will occur over HTTP requests, facilitating real-time data access for features like location services and weather updates.

## 4. System Features

### 4.1 User Sign Up

| | | | |
|---|---|---|---|
| Use Case ID: | AC01 | | |
| Use Case Name: | Sign Up | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |

| Date Created: | 01/09/2024 | Date Last Updated: | 01/09/2024 |
|---|---|---|---|

| | |
|---|---|
| Actor: | User, Email Service |
| Description: | Verifies the user's email and creates a password protected account linked to the email, which will be identified using an unique userID |
| Preconditions: | 1. User must have access to their email<br>2. Application must not have an existing user logged in<br>3. The databases must be online<br>4. The application must be able to communicate with databases |
| Postconditions: | |
| Priority: | High |
| Frequency of Use: | Medium |
| Flow of Events: | 1. User clicks the "Sign Up" button in login page<br>1.1. User proceeds to sign up page<br>2. User input Name, Email address, Password and Weight<br>3. User clicks on "send OTP" button.<br>3.1. Email service sends an OTP to email address<br>3.2. Application notify user to verify their email by inputting OTP sent to email.<br>4. User input OTP.<br>5. User clicks the "Sign Up" button in signup page<br>6. If OPT is correct:<br>6.1. Application generates an account with unique userID in Account Database and APP Database<br>6.2. Application assigns the Email Address and Password to the userID in the Account Database<br>6.3. Application assigns Name and Weight to the userID in the APP Database |

| | |
|---|---|
| | 7. User is logged in and proceeds to application main page |
| Alternative Flows: | AF-AC01-S5: If any input is missing<br>1. Application prompts user to input corresponding missing input<br>2. User proceeds to step 2.<br><br>AF-AC01-S6: If OTP is incorrect<br>1. Application prompts user "Incorrect OTP entered. Try again."<br>2. Application clears the OTP text field.<br>3. User proceeds to step 3. |
| Exceptions: | EX-AC01-01: If email already associated with an account in Account database<br>1. Application notifies user that an account has been registered with this email.<br>2. User proceeds to step 2. |
| Includes: | |
| Special Requirements: | |
| Assumptions: | 1. User installed the application<br>2. User's device is connected to internet via wifi or cellular data |
| Notes and Issues: | |

**Functional Requirement**

1. The system must have a "Login" button on the Sign Up page that navigates to the Login Page
2. The system must display four input text field in the Sign Up page: "Name", "Email Address", "Password", "OTP", and "Weight".
    2.1. All input fields submitted by user must not be empty
    2.2. The password must be at least 8 character long and includes atleast 1 special character and atleast 1 upper case

2.3.The system must display a “Send OTP” button beside the OTP input field
2.3.1. Each OTP sent is valid for 5 minutes
2.3.2. The OTP must not be sent more than once within 60 seconds
2.3.3. The previous OTP must be inactivated when a new OTP is sent

3. The system must validate user’s email address using an OTP sent to the submitted email address
3.1. If the OTP entered is incorrect, the system should:
3.1.1. Prompt the user: “Incorrect OTP entered. Try again.”
3.1.2. Clear the OTP text field
4. The system must generate an unique user ID linked to the newly created account
5. The users should be logged in after a successful sign up, and directed to the application main page

## 4.2 User Login

| | | | |
|---|---|---|---|
| Use Case ID: | AC02 | | |
| Use Case Name: | User Login | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 01/09/2024 | Date Last Updated: | 01/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Login to application with user’s account email and password |
| Preconditions: | 1. Application must not have an existing user logged in<br>2. User’s account email must be verified and linked to an existing account in the database |

| | |
|---|---|
| Postconditions: | - |
| Priority: | High |
| Frequency of Use: | medium |
| Flow of Events: | 1. User input email address and password<br>2. User clicks the “login” button<br>3. The application passes email address and password to AC03 to verify login credentials<br>4. If AC03 return TRUE, the user is logged in successfully<br>5. User proceeds to application main page |
| Alternative Flows: | AF-AC02-S2: If any input is missing<br>1. Application prompts user to input corresponding missing input<br>2. User proceeds to step 1.<br>AF-AC-02-S4: If AC03 return FALSE<br>1. Application notifies user that login has failed<br>2. Application returns to step 1. |
| Exceptions: | |
| Includes: | AC03, AC07 |
| Extends: | AC01 |
| Special Requirements: | |
| Assumptions: | |

| | |
|---|---|
| Notes and Issues: | |

**Functional Requirement**

1. The system must have a “Sign Up” button on the login page, that navigates to the Sign Up page
2. The system must have a “Forget Password” button on the login page, that navigates to the Forget Password floating tab.
3. The system must display two input text field in the Login page: “Email Address and “Password”
    3.1. All input fields submitted must not be empty
4. If the login credentials entered is/are incorrect, the system must:
    4.1. Prompt the user “Incorrect Email or Password entered!”
    4.2. Clear both input fields
5. The system should navigate to the application main page upon successful login

## 4.3 Verify Credentials

| | | | |
|---|---|---|---|
| Use Case ID: | AC03 | | |
| Use Case Name: | Verify Credentials | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 01/09/2024 | Date Last Updated: | 01/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Verifies the user’s account email and password and return TRUE or FALSE |

| | |
|---|---|
| Preconditions: | 1. The database must be online<br>2. The application must be able to communicate with database |
| Postconditions: | - |
| Priority: | High |
| Frequency of Use: | Medium |
| Flow of Events: | 1. If email address is passed in:<br>1.1. The application verifies Email Address against Password with Account Database<br>2. If credentials are valid<br>2.1. Return TRUE. |
| Alternative Flows: | AF-AC03-S1: If userID is passed in:<br>1. The application verifies userID against Password with Account Database<br>AF-AC03-S2: If login credentials are invalid<br>1. Return FALSE |
| Exceptions: | |
| Includes: | |
| Special Requirements: | |
| Assumptions: | 1. User's device is connected to internet via wifi or cellular data |
| Notes and Issues: | |

**Functional requirements**

1. The system must verify the email address against password with the account database upon receiving an email address for verification.

   The system must

   1.1 check if the provided email exists in the database.

   1.2 validate if the provided password matches the password associated with the email in the database

   1.3 return true if the email and password combination is valid

2. The system must verify the userID against password with the account database upon receiving an userID for verification.
3. The system should return TRUE if the email address and password combination is valid.

## 4.4 User Logout

| | | | |
|---|---|---|---|
| Use Case ID: | AC04 | | |
| Use Case Name: | User Logout | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 01/09/2024 | Date Last Updated: | 01/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Logout the user account and return to login page |
| Preconditions: | 1. User must be logged in |

| | |
|---|---|
| Postconditions: | |
| Priority: | High |
| Frequency of Use: | Medium |
| Flow of Events: | 1. User clicks on “logout” button<br>2. User is logged out and proceeds to login page |
| Alternative Flows: | |
| Exceptions: | |
| Includes: | |
| Special Requirements: | |
| Assumptions: | |
| Notes and Issues: | |

**Functional requirements**

1. The system must display a “logout” button on the user’s Self profile page
2. The users must be logged out upon clicking on the ‘logout’ button on the profile page.
3. The system must display the login page, if no user is logged in.

## 4.5 Delete Account

| | | | |
|---|---|---|---|
| Use Case ID: | AC05 | | |
| Use Case Name: | Delete Account | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 01/09/2024 | Date Last Updated: | 01/09/2024 |

| | |
|---|---|
| Actor: | User, Email Service |
| Description: | |
| Preconditions: | 1. User must be logged in |
| Postconditions: | 2. User account deleted from Account Database and App Database |
| Priority: | Low |
| Frequency of Use: | Rare |
| Flow of Events: | 1. User clicks on “Delete Account” button in edit account page<br>2. Application display a floating tab, notify user to confirm account deletion by inputting an OPT sent to email.<br>2.1. Email service sends an OTP to email address<br>3. User input OTP.<br>4. User clicks the “Delete Account” button in floating tab<br>5. If OPT is correct:<br>5.1. The user account is deleted from Account Database and App Database<br>5.2. Application notifies user that account has been deleted, and redirects to login page after 10 seconds. |

| | |
|---|---|
| Alternative Flows: | AF-AC05-S4: If OTP input is missing<br>1. Application prompts user to input corresponding missing input<br>2. User proceeds to step 3.<br><br>AF-AC05-S5: If OPT is incorrect<br>1. Application prompts user "Incorrect OPT"<br>2. Application closes the floating tab.<br>3. User proceeds to step 1. |
| Exceptions: | |
| Includes: | |
| Special Requirements: | |
| Assumptions: | 1. User has access to the email associated with their account<br>2. The connection of the application and Account Database and App Database are established |
| Notes and Issues: | |

**Functional Requirement**

1. The system must display a "Delete Account" button on
    1.1. The edit account page, and
    1.2. The delete account confirmation floating tab.
2. The floating tab must have
    2.1. One input text field taking OTP input, and a "Cancel" button
3. The system must send an OTP to the email address linked to the user's account
    3.1.Each OTP sent is valid for 5 minutes
4. If the OTP entered is incorrect, the system should:
    4.1. Prompt the user: "Incorrect OTP entered."
    4.2. Closes the floating tab.

5. The user's data must be deleted from both APP and Account database upon successful account deletion.

## 4.6 Change Password

| | | | |
|---|---|---|---|
| Use Case ID: | AC06 | | |
| Use Case Name: | Change Password | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 02/09/2024 | Date Last Updated: | 02/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Update new password linked to the user account |
| Preconditions: | 1. User must be logged in |
| Postconditions: | 1. User password is updated |
| Priority: | Medium |
| Frequency of Use: | Low |
| Flow of Events: | 1. User input "Current Password", "New Password" and "Retype Password".<br>2. User clicks on "Change Password" button<br>3. If "New Password" matches "Retype Password", user proceed to step 4. |

| | |
|---|---|
| | 4. Application passes userID and Current Password to AC03 to verify the user.<br>5. If AC03 returns TRUE:<br>5.1. Application updates the New Password into Account Database<br>5.2. Application notifies user that password is updated<br>6. User proceeds to Profile page |
| Alternative Flows: | AF-AC06-S3: If "New Password" does not match "Confirm New Password"<br><br>1. Application prompts user that passwords does not match<br>2. Application clears the New Password and Confirm New Password text fields<br>3. User proceeds to step 1.<br><br>AF-AC06-S5: If AC03 returns FALSE<br><br>1. Application prompts user "Incorrect Password entered. Try again."<br>2. Application clears the input text field.<br>3. User proceeds to step 1. |
| Exceptions: | |
| Includes: | AC03 |
| Special Requirements: | |
| Assumptions: | 1. The connection of the application and Account database is established |
| Notes and Issues: | |

**Functional Requirement**

1. The system must display a "Change Password" button on
   1.1. The edit account page, and

1.2. The change password page.

2. The change password page must have
    2.1. Three input text field taking current password, new password, and confirm new password input
    2.2. All input fields must not be empty
3. The system must validate if current password is valid
4. The system must validate if new password matches with confirm new password
5. If requirement 3 and 4 is/are incorrect, the system must:
    5.1. Prompt the user “Incorrect Password entered!”, and/or
    5.2. “New password does not match!”
    5.3. Clear all input fields
6. User must returns to their profile page upon successful password change.

## 4.7 Forget Password

| | | | |
|---|---|---|---|
| Use Case ID: | AC07 | | |
| Use Case Name: | Forget Password | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 19/09/2024 | Date Last Updated: | 19/09/2024 |

| | |
|---|---|
| Actor: | User, Email Service |
| Description: | Update new password linked to the user account |
| Preconditions: | 1. User input email address must be linked to a valid account in Account Database |
| Postconditions: | 2. A temporary password is generated |

| | |
|---|---|
| Priority: | Medium |
| Frequency of Use: | Low |
| Flow of Events: | 1. User click “forget password” button on login page<br>2. Application display floating tab, requesting for email address<br>3. User enter email address<br>4. User click send “Confirm” button<br>5. Application notify user that temporary password is sent and, floating tab closes. |
| Alternative Flows: | AF-AC07-S4: If email not found in Account Database<br>1. Application prompt user email not found.<br>2. User proceed to Step 2. |
| Exceptions: | EX-AC07-01: If user click “Cancel” button<br>1. Floating tab closes. |
| Includes: | |
| Special Requirements: | |
| Assumptions: | 2. The connection of the application and Account database is established |
| Notes and Issues: | |

**Functional Requirement**

1. The system must validate if the email address is linked to an existing account
    1.1. The system must prompt user if an invalid email address is entered
    1.2. The input field is cleared

2. The forget password floating tab must display:
    2.1. One input text field: “Email Address”
    2.2. Cancel and Confirm buttons
    2.3. All input fields must not be empty
3. The new temporary password generated must be:
    3.1. At least 8 character long and includes atleast 1 special character and atleast 1 upper case
    3.2. Only valid for 3 days if unused.
    3.3. Replaced as new password upon first time login with it
4. Previous password must remains active until first time login with temporary password sent.
5. The floating tab must be closed when user click on “Cancel” button.

## 4.8 Edit Account

| | | | |
|---|---|---|---|
| Use Case ID: | AM01 | | |
| Use Case Name: | Edit Account | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 02/09/2024 | Date Last Updated: | 02/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Update account information (“Name”, “Bio”, “Weight”) |
| Preconditions: | 1. User must be logged in |
| Postconditions: | |

| | |
|---|---|
| Priority: | Medium |
| Frequency of Use: | Medium |
| Flow of Events: | 1. Application loads the input fields with the user's account information.<br>2. User updates the account information input fields.<br>3. User clicks "Save Changes" button<br>4. Application updates input into APP Database<br>5. User proceeds to Profile page |
| Alternative Flows: | AF-AM01-S3: If user clicks "Save Changes" button and "Name" field is empty<br><br>1. Application prompt user to input a Name<br>2. User proceeds to step 2.<br><br>AF-AM01-S3: If user clicks "Save Changes" button and "Weight" field is empty<br><br>3. Application prompt user to input a Weight<br>4. User proceeds to step 2.<br><br>AF-AM01-S3: If user clicks on "Delete Account" button<br><br>1. User proceeds to AC05<br><br>AF-AM01-S3: If user clicks on "Change Password" button<br><br>1. User proceeds to AC06 |
| Exceptions: | |
| Includes: | |
| Extends: | AC05, AC06 |
| Special Requirements: | |

| | |
|---|---|
| Assumptions: | 1. The connection of the application and APP database is established |
| Notes and Issues: | |

**Functional Requirement**

1. The system must display a “Edit Profile” button on the user’s self profile page
2. The edit account page must have:
    2.1. Three text input field for Name, Bio, and Weight
    2.2. Change Password, Delete Account, and Save Changes Buttons.
3. The system must fill up the input field with their respective existing datas.
4. If name and weight are empty when user clicked on “save changes” button
    4.1. The system must prompt user that name and weight field must not be empty
    4.2. Changes must not be saved
5. User must returns to their profile page after successfully saving their changes.

## 4.9 View Self Profile

| | | | |
|---|---|---|---|
| Use Case ID: | AM02 | | |
| Use Case Name: | View Self Profile | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 02/09/2024 | Date Last Updated: | 05/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Loads the account data and constructs the profile page for users (Self) |
| Preconditions: | 1. User must be logged in |

| | |
|---|---|
| Postconditions: | |
| Priority: | High |
| Frequency of Use: | Very High |
| Flow of Events: | 1. User clicks on “Profile” button on menu bar<br>2. Application retrieve and loads the user’s profile page from APP database<br>3. If user clicks on “Edit Account” button in “Profile” page<br>3.1.User proceeds to AM01 |
| Alternative Flows: | AF-AM02-S3: If user clicks on “Logout” button in “Profile” page<br>1. User proceeds to AC04 |
| Exceptions: | |
| Includes: | |
| Extends: | AC04, AM01 |
| Special Requirements: | |
| Assumptions: | 1. The connection of the application and APP database is established |
| Notes and Issues: | |

**Functional Requirement**

1. The system must display on user self profile page:
    1.1. User’s Name, User ID, Email Address, Bio, Badges, and

1.2. User's statistics:

1.2.1. WeightTotal Carbon Saved, and Total Calorie Burnt

2. The profile page must have the Edit Profile and Logout buttons

## 4.10 View Friend Profile

| | | | |
|---|---|---|---|
| Use Case ID: | AM03 | | |
| Use Case Name: | View Friend Profile | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 02/09/2024 | Date Last Updated: | 05/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Loads the account data and constructs the profile page for users (Friend) |
| Preconditions: | 2. User must be logged in |
| Postconditions: | |
| Priority: | High |
| Frequency of Use: | Very High |
| Flow of Events: | 1. User clicks on "Profile" button beside friend name in friend list<br>2. Application retrieve and loads the friend's profile page from APP database<br>3. If user clicks on "Delete Friend" button |

| | |
|---|---|
| | 3.2.User proceeds to FR02 |
| Alternative Flows: | AF-AM03-S1: User clicks on the friend's Name Tab in friend list<br>1. User proceed to step 2. |
| Exceptions: | |
| Includes: | |
| Extends: | FR02 |
| Special Requirements: | |
| Assumptions: | 2. The connection of the application and APP database is established |
| Notes and Issues: | Email address and weight are Not shown on "Friend" page |

**Functional Requirements**

1. The system must display on friend's profile page:
    1.1. Friend's Name, User ID, Bio, Badges, and
    1.2. Friend's statistics:
        1.2.1. Total Carbon Saved, and Total Calorie Burnt
2. The system must not display friend's email address, and weight.
3. The friend's profile page must have the Delete Friend button

## 4.11 View Others Profile

| | |
|---|---|
| Use Case ID: | AM04 |
| Use Case Name: | View Others Profile |

| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
|---|---|---|---|
| Date Created: | 02/09/2024 | Date Last Updated: | 05/09/2024 |

| Actor: | User |
|---|---|
| Description: | Loads the account data and constructs the profile page for users (Others) |
| Preconditions: | 3. User must be logged in |
| Postconditions: | |
| Priority: | High |
| Frequency of Use: | Very High |
| Flow of Events: | 1. User clicks on other user's Name Tab in search result list<br>2. Application retrieve and loads the other user's profile page from APP database<br>3. If user had sent friend request to other user, application display "Requested button" |
| Alternative Flows: | AF-AM04-S3: If user did not send friend request to other user<br>1. Application display "Request" button<br>2. If user clicks on "Request" button<br>2.1.User proceeds to Step 6 of FR03 |
| Exceptions: | EX-AM04-01: If user received a friend request from other user<br>1. Application retrieve and loads the other user's profile page from APP database<br>2. Application displays, "Accept" and "Reject" button |

| | |
|---|---|
| | 2. User proceeds to Step 3 of FR04 |
| Includes: | |
| Extends: | FR03, FR04 |
| Special Requirements: | |
| Assumptions: | 3. The connection of the application and APP database is established |
| Notes and Issues: | Email address and weight are only shown on “My Profile” page |

**Functional Requirements**

1. The system must display on other user’s profile page:
    1.1. Name, User ID, Bio, Badges, and
    1.2. Statistics:
        1.21. Total Carbon Saved, and Total Calorie Burnt
2. The system must not display other user’s email address, and weight.
3. If user has not sent friend request to this other user, the other user’s profile page must have the Request button
4. If user sent friend request to this other user, the other user’s profile page must have the Requested button
5. If user received friend request from this other user, the other user’s profile page must have Accept and Reject buttons

## 4.12 Wellness Zone Recommendation

| | |
|---|---|
| Use Case ID: | WZ01 |
| Use Case Name: | Wellness Zone Recommendation |

| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
|---|---|---|---|
| Date Created: | 01/09/2024 | Date Last Updated: | 18/09/2024 |

| Actor: | User, NParks API, Google Map API |
|---|---|
| Description: | 1. Search for wellness zones around the user and display a recommended list, adjustable by radius |
| Preconditions: | 1. User must be logged in<br>2. GPS services must be activated |
| Postconditions: | 1. User is able to view a list of wellness zone recommendation (ie. green spaces).<br>2. The address can be pushed to Select Trip (ST01) when required. |
| Priority: | Medium |
| Frequency of Use: | Medium |
| Flow of Events: | 1. User navigate to “Wellness” option on menu bar<br>2. Application establish connection to user’s GPS and Npark API<br>3. Application push out a map and a list of locations that is green zone/healthy corners using google map API<br>3.1.System defaults the recommendation radius at 5km<br>4. User select a wellness zone recommended in the list<br>5. Application display the details of the selected wellness zone<br>5.1. Application loads Weather information concurrently with WF01.<br>6. User select “Go” button on the wellness zone location that they wants to go<br>7. Application will take down the address of the destination and bring user to Trip Mehod (ST01) for further actions |

| | |
|---|---|
| Alternative Flows: | AF-WZ01-S3.1: If user changes recommendation radius<br>1. Application goes to step 3, with step 3.1 recommendation radius set at user's choice<br>AF-WZ01-S5: If user select another wellness zone<br>1. User return back to step 4. |
| Exceptions: | If there are no GPS services, User will be prompt to turn on their GPS service. |
| Includes: | ST01, WF01 |
| Special Requirements: | GPS services must be activated. |
| Assumptions: | The connection of the application, API and Database are established |
| Notes and Issues: | |

**Functional Requirement**

1. The system must display a list of wellness zone recommendations to user
    1.1. Each wellness zone must displayed
        1.1.1. A pin over the google map API
        1.1.2. Name of location, Brief description, Distance from user
        1.1.3. "Go" button, navigating the user to plan their trips to their selected wellness zone in the Maps page.
    1.2. If selected the pin of the wellness zone must be enlarged
2. The system must display a sliding bar to allow user choose recommendation radius
    2.1.Radius ranges between 5 - 50 km radius range, and must be integers.
    2.2.Radius must be default at 5km
3. When selected the 2-hour weather information must be overlay at the middle-top of the map as a floating widgit.

## 4.13 Manage Friend List

| | | | |
|---|---|---|---|
| Use Case ID: | FR01 | | |
| Use Case Name: | Manage Friendlist | | |
| Created By: | Chen Ziyan | Last Updated By: | Tan Yi Da |
| Date Created: | 02/09/2024 | Date Last Updated: | 05/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | An option in the friend menu that show a list of friends and give the user option to add new friends or manage incoming friend request |
| Preconditions: | 1. User is logged in |
| Postconditions: | 1. User can view their friend list |
| Priority: | medium |
| Frequency of Use: | Medium |
| Flow of Events: | 1. User navigate to the “Friends” page of the application<br>2. Application display a list of current friends<br>3. If user clicks on “Profile” button beside friends’s name, user proceed to view friend profile using AM03 |
| Alternative Flows: | AF-FR01-S3: If user clicks on “Add Friends” button in “Friends” page |

| | |
|---|---|
| | 1. User proceeds to "Add Friend" sub-page of Friend page<br>1.1. Applications proceeds to FR03 and FR04 |
| Exceptions: | EX-FR01-01: If user has no friends<br>1. Application displays "You have no active friends on ! Add your friends now!".<br>2. Application remains on the same page |
| Includes: | AM03, FR03, FR04 |
| Extends: | |
| Special Requirements: | System must ensure that friend request and securely processed and that user privacy is maintained |
| Assumptions: | User has active internet connection to access and manage their friend list |
| Notes and Issues: | Ensure that friend list is updated in real time each time the user pressed on FR01 |

**Functional Requirement**

1. Users must be able to view the list of his friends
2. The BottomMenuUI must have a "Friends" button to view his friend list
    a. The application must display a list of all user's friends
3. d

REQ29.2.

REQ29. Users must be able to view the profile of a friend

REQ30.1. The application must have a "Profile" button beside each and every friend's name

REQ30.2 The application must show the friend's profile after pressing on "Profile"

REQ30. User must be able to add friends or manage incoming friend request

REQ31.1 The application must have a "Add Friends" button for the mentioned function

## 4.14 Remove Friend

| | | | |
|---|---|---|---|
| Use Case ID: | FR02 | | |
| Use Case Name: | Remove Friend | | |
| Created By: | Chen Ziyan | Last Updated By: | Tan Yi Da |
| Date Created: | 03/09/2024 | Date Last Updated: | 18/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | A function for user to remove a friend from his friend list |
| Preconditions: | 1. User must be logged in<br>2. User must have at least 1 friend in their friend list<br>3. User is in View Friend Profile (AM03) of the friend to be removed |
| Postconditions: | 1. The selected friend is removed from the user's friend list<br>2. The user's friend list is updated, user will be able to see the updated list once he press Manage Friend List (FR01) use case again |
| Priority: | Medium |
| Frequency of Use: | Low |
| Flow of Events: | 1. Application prompt user to confirm the removal with an floating tab<br>2. If user clicks "Delete" button |

| | |
|---|---|
| | 3. The application remove the selected friend from the user's friend list<br>4. Application will return to FR01 after successful removal of selected friend |
| Alternative Flows: | AF-FR02-02: If user clicks "Cancel" button<br>1. Application closes the floating tab. |
| Exceptions: | - |
| Includes: | AM03, FR01 |
| Special Requirements: | The system must ensure that the removal of a friend is securely processed and that user privacy is maintained |
| Assumptions: | User has active internet connection |
| Notes and Issues: | Need to implement a confirmation step to prevent accidental removal of friends |

**Functional Requirement**

REQ32. The user must be able to select a friend from his friend list to remove.

REQ32.1. The application must display the profile of the friend after pressing on the "Profile" button.

REQ32.2. The ProfileUI of the friend must include a "Delete Friend" button to remove the friend from the user's friend list.

REQ32.3. The application must have a secondary confirmation on each removal of a friend

REQ32.3.1. ConfirmFriendRemovalUI must have option "Confirm" and "Cancel"

REQ33. The system must update the friend list if the user removes a friend from the friend list.

REQ34. Pop up "confirm delete friend"

## 4.15 Add Friend

| Use Case ID: | FR03 | | |
|---|---|---|---|
| Use Case Name: | Add Friend | | |
| Created By: | Chen Ziyan | Last Updated By: | Chen Ziyan |
| Date Created: | 03/09/2024 | Date Last Updated: | 03/09/2024 |

| Actor: | User |
|---|---|
| Description: | Allow user to add a new friend to their friend list |
| Preconditions: | 1. User must be logged in<br>2. User has access to the friend's UID |
| Postconditions: | 1. A friend request is sent to the selected user |
| Priority: | medium |
| Frequency of Use: | medium |
| Flow of Events: | 1. User selected "Add Friend" button in Friend page previously in FR01<br>2. Application prompt user to search the UID of the user they wish to add<br>3. User enters the required information<br>4. Application show user the name that is linked to the entered UID<br>5. User opens the other user's profile using AM03 |

| | |
|---|---|
| | 6. If user clicks on “Request” button previously in AM03<br>7. Application sends the friend request to the other user<br>8. Application refresh the page using AM03. |
| Alternative Flows: | |
| Exceptions: | If entered UID is not valid or does not exist in the system, system display an error message and prompt user to enter a valid UID |
| Includes: | AM03 |
| Special Requirements: | The system must ensure that friend requests are securely processed and that user privacy is maintained |
| Assumptions: | User has active internet connection |
| Notes and Issues: | Need to implement a confirmation step to prevent user to add a wrong friend |

**Functional requirements**

REQ34. The user should be able to send a friend request to the other users by searching the user ID.

REQ35. If the input user ID is incorrect, the system shall prompt the user to input again.

REQ36. The system shall update the friendlist if the user adds a friend to the friendlist.

## 4.16 Manage Friend Request

| | | | |
|---|---|---|---|
| Use Case ID: | FR04 | | |
| Use Case Name: | Manage friend request | | |
| Created By: | Chen Ziyan | Last Updated By: | Tan Yi Da |
| Date Created: | 03/09/2024 | Date Last Updated: | 05/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Allow user to manage incoming friend requests |
| Preconditions: | 1. User must be logged in |
| Postconditions: | 1. Incoming friend request will be either:<br>1.1. Accepted<br>1.1.1. Friend is notified that they have became friends with user.<br>1.2. Rejected |
| Priority: | medium |
| Frequency of Use: | medium |
| Flow of Events: | 1. User selected “Add Friend” button in Friend page previously in FR01<br>2. Application will show the user a list of incoming friend requests |

| | |
|---|---|
| | 3. User select “Accept” button to add the friend into his friend list, now both users will have each other on their friend list.<br>4. Application send a notification to the sender of the friend request upon successfully added the User. |
| Alternative Flows: | AF-FR04-S3: User select “Reject” button<br>1. Friend requests will be canceled |
| Exceptions: | - |
| Includes: | - |
| Special Requirements: | 1. The sender of the friend request will not be notified if request is rejected |
| Assumptions: | - |
| Notes and Issues: | - |

**Functional requirements**

REQ37. After searching for the userID of the other user, the user must be able to send a request to the other users to become friends.

REQ38. The user must be able to choose to accept or reject the request from any other users.

REQ38.1. The user should click on ‘add’ to accept a request and ‘reject’ to reject a request.

REQ39. If the user accepts the request, then the one who sends the request shall be added to the user’s friendlist.

REQ40. If the user rejects the request, then the one who sends the request shall receive a message displaying that the request has been rejected.

## 4.17 Display Rank

| | | | |
|---|---|---|---|
| Use Case ID: | DL01 | | |
| Use Case Name: | Display Rank | | |
| Created By: | Chen Ziyan | Last Updated By: | Chen Ziyan |
| Date Created: | 01/09/2024 | Date Last Updated: | 01/09/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | Display a scrollable leaderboard for everyone that the user has added as friends (including the user) |
| Preconditions: | 1. The user must be logged in<br>2. The user must have existing friends |
| Postconditions: | Show the user the list with different filters available<br>(ie most CO2 saved, Most calory burnt etc) |
| Priority: | Medium |
| Frequency of Use: | Frequent |
| Flow of Events: | 1. The user select the Rank button on the menu bar<br>2. The application will show the user by default the ranking of the friends based on “Most CO2 Saved”<br>3. The application will highlight the user’s record |

| | |
|---|---|
| | 4. The user can select 1 other filters such as “Most calory burnt”<br>5. The application will show the user the ranking of the friends based on “Most calory burnt” |
| Alternative Flows: | AF-DL01-04: user select another different filter<br>1. User proceed to step 5, showing ranking for that 1 chosen filter |
| Exceptions: | Ex-01: If User has no friends, the application will show a floating window to add friends. Once the User pressed on “X” on the floating window, the User will only see his own name on the leaderboard. |
| Includes: | |
| Special Requirements: | 1. The application will reset the rank weekly<br>2. Maximum of 1 filter can be selected each time |
| Assumptions: | The connection of the app and the Database is established |
| Notes and Issues: | |

Functional requirement

REQ41. The system shall display ranks which consists of a calories burned leaderboard and a carbon saved leaderboard.

REQ42. Users must be able to switch between viewing the two leaderboards by clicking a button on the page.

REQ43. The system must also display the weekly amount of calories burned/carbon saved of each user on the leaderboards.

## 4.18 Select Trip

| Use Case ID: | ST01 | | |
|---|---|---|---|
| Use Case Name: | Select Trip | | |
| Created By: | Chen Ziyan | Last Updated By: | Lim Jun Jie |
| Date Created: | 01/09/2024 | Date Last Updated: | 09/11/2024 |

| Actor: | User, Google Map API |
|---|---|
| Description: | The user can search for a location based on keyboard input using this functionality. User inputs a location and the system will communicate with the Google Map API to predict and retrieve all locations that match the user's input. |
| Preconditions: | 1. The user must be logged in.<br>2. The User's current location must be known. |
| Postconditions: | The embedded Google Map shows the route from user's current location to the searched location. |
| Priority: | High |
| Frequency of Use: | Very Frequent |
| Flow of Events: | 1. User types a location in the search box.<br>2. The system communicates with the Google Map API. |

| | |
|---|---|
| | 3. Google Map API will retrieve the location based on the user's input and show the route from the user's current location to the searched location.<br>4. Application will prompt user to Select Trip Method (ST02) to select the travel method the user requires. |
| Alternative Flows: | |
| Exceptions: | Ex-01: User click "Cancel Trip" (ST05)<br>a. System return to Step 1 |
| Includes: | ST02 |
| Special Requirements: | |
| Assumptions: | |
| Notes and Issues: | |

Functional requirement

REQ44. The system must be able to know the current location of the user.

REQ45. The user must be able to search for location based on an input keyword.

REQ45.1. The system must provide an input field for user to input location.

REQ46. The system must be able to communicate with Google Map API to show the route from the user's current location to the searched location on the map.

## 4.19 Trip Method

| Use Case ID: | ST02 | | |
|---|---|---|---|
| Use Case Name: | Trip Method | | |
| Created By: | Chen Ziyan | Last Updated By: | Lim Jun Jie |
| Date Created: | 02/09/2024 | Date Last Updated: | 09/11/2024 |

| Actor: | User, Google Map API |
|---|---|
| Description: | The user must select the trip method |
| Preconditions: | 1. The user must be logged in<br>2. The User's current location must be known<br>3. A trip has been selected in ST01 |
| Postconditions: | Destination has confirmed |
| Priority: | Medium |
| Frequency of Use: | Very frequent |
| Flow of Events: | 1. After ST01, the route from User's current location to the selected location will be tracked.<br>2. Application will show other travel method as icons:<br>a. Walking<br>b. Motorcycle<br>c. Bus<br>d. Car |

| | |
|---|---|
| | 3. User selects the trip method for the trip.<br>4. The system displays the trip metrics as an overlay on the Google Map for each trip method selected. |
| Alternative Flows: | - |
| Exceptions: | - |
| Includes: | ST03, ST04, ST05 |
| Special Requirements: | - |
| Assumptions: | Destination entered by user is accurate |
| Notes and Issues: | - |

**Functional requirements**

REQ46. The user must select a trip method among walking, cycling, bus and car after the user has selected a trip.

REQ47. After the user selects a trip method, the system must display the route from the start location to the destination on a map.

REQ48. The system must display the direction during the trip based on the real-time location of the user.

## 4.20 Start Trip

| Use Case ID: | ST03 | | |
|---|---|---|---|
| Use Case Name: | Start Trip | | |
| Created By: | Chen Ziyan | Last Updated By: | Lim Jun Jie |
| Date Created: | 02/09/2024 | Date Last Updated: | 09/11/2024 |

| Actor: | User |
|---|---|
| Description: | After ST01, the user may start the trip and continue his journey. |
| Preconditions: | 1. The user must be logged in<br>2. The User's current location must be known<br>3. A trip method has been selected in ST02 |
| Postconditions: | 1. Destination has reached , or<br>2. User has pressed end trip (ST05) |
| Priority: | Medium |
| Frequency of Use: | Very frequent |
| Flow of Events: | 1. After ST02, User's location will be tracked.<br>2. The application will show the direction using Google Map API based on the user's real time location<br>3. Once the user has reached his destination **or** press end trip (ST05), trip's details will be calculated and saved in the database (DA01) |

| | |
|---|---|
| Alternative Flows: | - |
| Exceptions: | ST03-EX01: If User have not chosen either a location or travel method.<br>1. When the User clicks "Start' button, the user will display the message prompting user to enter select a location and travel method before starting the trip. |
| Includes: | DA01 |
| Special Requirements: | - |
| Assumptions: | Destination and trip method is selected accurately |
| Notes and Issues: | - |

**Functional requirements**

REQ49. The user must be able to start a trip after confirming the trip information (selected destination and trip method).

## 4.21 End Trip

| | | | |
|---|---|---|---|
| Use Case ID: | ST04 | | |
| Use Case Name: | End Trip | | |
| Created By: | Chen Ziyan | Last Updated By: | Lim Jun Jie |
| Date Created: | 01/09/2024 | Date Last Updated: | 09/11/2024 |

| | |
|---|---|
| Actor: | User, Google Map API |
| Description: | The user can manually end the trip after trip have been started. |
| Preconditions: | 1. The user must be logged in<br>2. The User's GPS location must be known<br>3. The user must have started a trip using the application |
| Postconditions: | 1. The trip is marked as completed<br>2. Trip details as recorded in App Database |
| Priority: | Medium |
| Frequency of Use: | Very Frequent |
| Flow of Events: | 1. User selects the "End Trip" option<br>2. Application prompts user to confirm the end of the trip<br>3. User confirm the end of the trip<br>4. Application update App Database with new trip details<br>5. Application proceed to DA01 |

| | |
|---|---|
| Alternative Flows: | - |
| Exceptions: | Ex-ST04-01: User click “Cancel Trip”<br>1. System proceed to ST05. |
| Includes: | ST05, DA01 |
| Special Requirements: | |
| Assumptions: | 1. User has the permission to end the trip<br>2. Application has the capacity to record and update trip history |
| Notes and Issues: | - |

**Functional requirements**

REQ50. The system must be able to detect if the user has arrived at the destination.
REQ51. The system should display an ‘end trip’ button on the page.
REQ52. User can end trip by clicking end button.
REQ53. System must redirect User to DA01 upon successful ending of trip.

## 4.22 Cancel Trip

| | | | |
|---|---|---|---|
| Use Case ID: | ST05 | | |
| Use Case Name: | Cancel Trip | | |
| Created By: | Chen Ziyan | Last Updated By: | Lim Jun Jie |
| Date Created: | 02/09/2024 | Date Last Updated: | 09/11/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | At any point in time, User is allowed to cancel the trip. Canceled trips will not be saved in the database, after cancellation, the application will bring the user back to ST01. |
| Preconditions: | 1. Trip has been selected, and/or<br>2. Trip has started<br>3. User must be logged in |
| Postconditions: | 1. Trip is marked as canceled<br>2. No trip details are recorded in the user's trip history |
| Priority: | Medium |
| Frequency of Use: | Frequent |
| Flow of Events: | 1. User select "Cancel Trip" (ST05)<br>2. Application prompt user to confirm the cancellation of the trip<br>3. User confirms the cancellation of the trip<br>4. Application stop tracking user's location |

| | |
|---|---|
| | 5. The application will bring user back to ST01 |
| Alternative Flows: | - |
| Exceptions: | - |
| Includes: | - |
| Special Requirements: | - |
| Assumptions: | 1. User has the necessary permission to cancel the trip<br>2. Application has the capacity to stop tracking and clear any trip data after cancellation |
| Notes and Issues: | - |

**Functional requirements**

REQ52. The system must allow the user to cancel a trip at any time during the trip.

REQ52.1. The system should display a ‘cancel trip’ button on the page during the trip.

REQ53. Once the user confirms canceling the trip, the system must stop tracking the location of the user and navigate back to the ‘Select Trip’ page.

## 4.23 Display Achievement Progress

| | | | |
|---|---|---|---|
| Use Case ID: | DA01 | | |
| Use Case Name: | Display Achievement | | |
| Created By: | Chen Ziyan | Last Updated By: | Lim Jun Jie |
| Date Created: | 04/09/2024 | Date Last Updated: | 09/11/2024 |

| | |
|---|---|
| Actor: | User |
| Description: | After completion of the trip, Application will show user his progress and achievement badges after adding on the stats of the completed trip. |
| Preconditions: | 1. User has just ended a trip previously in ST04 |
| Postconditions: | Achievements and progress is shown to the user |
| Priority: | High |
| Frequency of Use: | High |
| Flow of Events: | 1. User has ended a trip previously in ST04.<br>2. Application computes the user's achievement progress with the new trip details.<br>3. Application displays the addition of the newly add progress of both calorie burnt and carbon saved on the progress bar.<br>4. User click on "Back to map" button, application return to ST01. |

| | |
|---|---|
| Alternative Flows: | - |
| Exceptions: | - |
| Includes: | - |
| Special Requirements: | - |
| Assumptions: | 3. User has the necessary permission to cancel the trip<br>4. Application has the capacity to stop tracking and clear any trip data after cancellation |
| Notes and Issues: | - |

**Functional requirements**

REQ54. After a trip has ended (not including the situation when a trip is canceled), the system shall calculate and display the total amounts of carbon saved and calories burned to the user (including the calculation of the latest trip).

REQ55. The system must be able to track the calories burnt and carbon saved for that specific trip

REQ56. The system must be able to add the relevant points to the respective progress bar for both calorie burnt and carbon saved.

REQ57. The user must hit certain predefined thresholds to obtain achievement badges.

REQ58. The system must be able to retain and retrieve the achievement progress earned via the database throughout the user's usage of the application before deletion.

## 4.24 Display Notification

| | |
|---|---|
| Use Case ID: | DN01 |
| Use Case Name: | Display Notification |

| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
|---|---|---|---|
| Date Created: | 05/09/2024 | Date Last Updated: | 05/09/2024 |

| Actor: | User |
|---|---|
| Description: | Displays a list of notification updates to the user and navigates the user to the corresponding page |
| Preconditions: | 1. User must be logged in<br>2. User has new updates on their account |
| Postconditions: | |
| Priority: | medium |
| Frequency of Use: | High |
| Flow of Events: | 1. User clicks on the notification icon  <br>2. Application displays a floating list of new notifications to user<br>3. User clicks on an individual notification “TAB”<br>4. If notification type is Friend Request<br>4.1. User proceeds to the profile page of the requesting user using AM04 |
| Alternative Flows: | AF-DN01-S4: If notification type is New Friend<br>1. User proceeds to profile page of the new friend using AM03. |
| Exceptions: | EX-DN01-01: If no notification, application displays “You have no notifications!” |

| | |
|---|---|
| Includes: | |
| Extends: | AM03, AM04 |
| Special Requirements: | |
| Assumptions: | 1. The connection of the application and APP database is established |
| Notes and Issues: | 1. Alternative flows to be added if more notification types are introduced in future |

**Functional requirements**

REQ59. The user must be able to view all the friend requests from others by clicking on the notification icon.

REQ60. Once the user clicks on any request, the system should navigate the user to the according friend profile page in order to accept/reject the request.

## 4.25 Weather Forecast

| | | | |
|---|---|---|---|
| Use Case ID: | WF01 | | |
| Use Case Name: | Weather Forecast | | |
| Created By: | Tan Yi Da | Last Updated By: | Tan Yi Da |
| Date Created: | 05/09/2024 | Date Last Updated: | 18/09/2024 |

| | |
|---|---|
| Actor: | User, NEA API |

| | |
|---|---|
| Description: | Displays a the weather details for a selected wellness zone in the recommendation list |
| Preconditions: | 1. User must be logged in |
| Postconditions: | 1. Weather information is displayed as a floating tab in WZ01 step 4. |
| Priority: | medium |
| Frequency of Use: | High |
| Flow of Events: | 1. Application receives the wellness zone selected by user in WZ01.<br>2. Application request the corresponding weather information from NEA API<br>3. Application display the 2-hour weather information in a floating tab overlaying WZ01. |
| Alternative Flows: | |
| Exceptions: | EX-WF01-01: If no weather information available<br>1. Application display “Weather Unavailable” |
| Includes: | |
| Extends: | |
| Special Requirements: | |

| | |
|---|---|
| Assumptions: | 2. The connection of the application, API and Database are established |
| Notes and Issues: | |

**Functional requirements**

1. The weather widget must display:
    1.1. **Header:** Name of the wellness zone
    1.2. Two-hourly weather status
    1.3. **Icons:** cloudy, rain, sun, for the respective weather status
2. The weather widget background must be 50% translucent.

# 5. Other Nonfunctional Requirements

## 5.1 Performance Requirements

**Response Time:**

Each page must load within 2 seconds.

The application must load wellness zones and calculate routes within 2 seconds of user input under normal operating conditions. For peak load conditions (e.g., during a national event), the response time should not exceed 5 seconds.

**Scalability:** The system should be capable of scaling to support up to 10000 simultaneous users without any degradation in performance.

**Data Processing:** The application should be able to process and display route calculations, carbon savings, and calorie burn data within 2 seconds.

## 5.2 Safety Requirements

**User Data Protection:** The system must prevent any unauthorized access or loss of personal data. In the event of a system failure, data recovery processes should ensure no loss of critical user information.

**Password Encrpytion:** The system must apply password encryption to all user credentials.

## 5.3 Security Requirements

**Access Control:** The system must implement role-based access control (RBAC) to restrict access to sensitive data and administrative functions.

**Data Privacy Compliance:** The application must comply with the Personal Data Protection Act (PDPA) in Singapore, ensuring that users' privacy is respected and their data is handled according to legal requirements.

## 5.4 Software Quality Attributes

**Availability:** The application should have an availability of 99.9%, ensuring that it is accessible to users at all times, with minimal downtime for maintenance.

**Correctness:** All calculations (carbon saved, calories burned) must be accurate.

**Interoperability:** The application should seamlessly integrate with third-party services, ensuring smooth data exchange.

**Maintainability:** The codebase should be modular and well-documented, allowing for easy updates and bug fixes by *Calowin* developers.

**Portability:** The application should be easily portable across different mobile operating systems, including iOS and Android, without requiring significant changes.

**Reliability:** The system should consistently perform its intended functions without failures, especially during peak usage times.

**Robustness:** The application should handle unexpected inputs or conditions gracefully, without crashing or producing incorrect outputs.

**Usability:** The user interface should be intuitive and easy to navigate, enabling users of all technical skill levels to use the application effectively.

### 5.5 Business Rules.

**Data Retention:** User activity data, such as trip history and wellness zone visits, should be retained for a minimum of one year to allow users to track long-term progress.

**User Consent:** Users must provide explicit consent before any personal data is shared with third parties, in compliance with data protection laws.

# 6. Other Requirements

# Appendix A: Glossary

## Data Dictionary

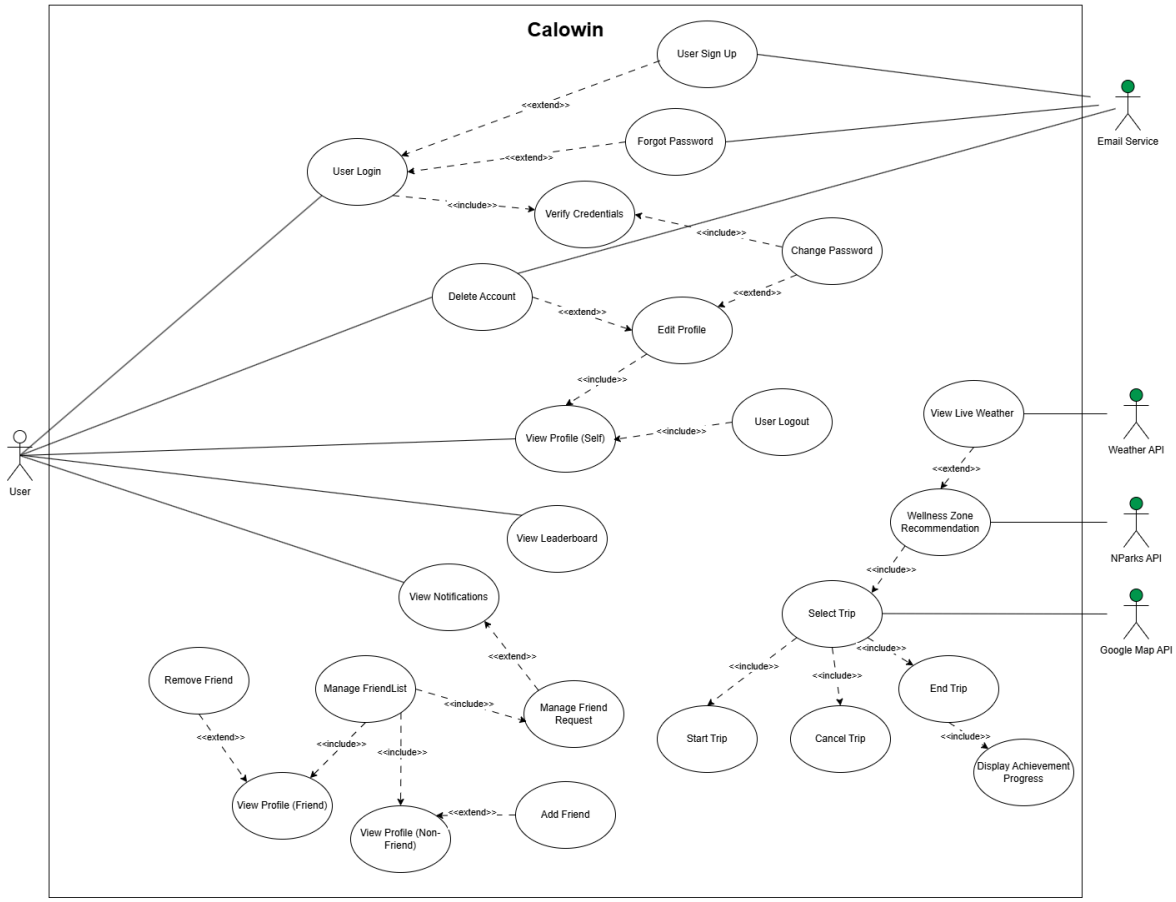
| Term | Definition |
|---|---|
| Application | Refers to the “Calowin” mobile application. |
| User | The entity that is using the mobile application after registering for an account. |
| Account Database | Refers to the Account database of the “CaloWin” mobile application, which stores all user-related information such as User ID, password, email. This database is separated from the App Database to enhance security |

| | |
|---|---|
| | within the application. |
| App Database | Refers to the App database of the “CaloWin” mobile application, which stores all the information of the application. This database contains dynamic user data, including trip metrics (such as calories burned and carbon saved), and achievements belonging to the users. This data is updated in real-time based on user actions. |
| Google API | An Application Programming Interface which is used for map integration in the application. |
| NParks API | An Application Programming Interface which is called to extract various National Parks in Singapore. |
| Weather API | An Application Programming Interface which is called to extract live weather data, |
| Sign Up | Refers to a feature that allows the user to sign up for an account to use the features in the application. |
| One-time Password (OTP) | A six-digit combination code which is sent to the User via their registered email address. The OTP serves as an additional layer of security for the User in the scenario where they forget their login credentials. |
| Login | Refers to a feature which requires User to key in information consisting of an email, paired with a password which must be at least 6 characters and should include a combination of numbers, letters and special characters that is created during sign up. |
| User Profile | Contains the user's basic information, including Name, userID, Bio, Weight and Achievement Badges. |
| Edit Profile | A feature which allows users to update their personal information and particulars such as their Name, Bio. Weight, and Password. User can also delete and wipe their account information through this feature. |
| Friends | Refers to other registered users of the application who are connected to a user’s Friendlist. Users can view their friends' achievements, stats and rankings in a leaderboard. |

| | |
|---|---|
| Friends List | A list which allows users to view their friends. Users can make a friend request to another registered user through the search of their userID. The friend list is accessible for all users using the app. |
| Leaderboard | A ranking system which ranks users based on their total high score which can be filtered by either ‘Carbon Saved’ or ‘Calories Burned’. In the leaderboard, information such as points and achievement badges are displayed. |
| Achievement Badges | Badges awarded to users as rewards for various achievements. These achievements are unlocked by completing trips and progressing within the app. |
| Calories | A measurement of energy expenditure during physical activity. Calories burned are measured by the system based on the distance traveled as well as the mode of transportation used for the trip. |
| Carbon | A measurement of the reduction in carbon emissions when choosing eco-friendly transportation. Carbons saved are measured based on the distance traveled as well as mode of transportation used for the trip. |
| Trip | Refers to the user's movement from starting point A to destination point B. |
| Mode of Transports | Method of transportation in which one is using to travel from starting point A to destination point B, which includes public transportation, car, cycling and walking. |
| Trip Method | A feature of the application designed to allow users to plan and start their trips by selecting a preferred mode of transport. Based on the User’s trips and selected transportation method, the application automatically calculates and displays the carbon savings and calories burned at the end of that trip. |
| Wellness Zones | Zones which contain facilities such as parks, sports fields, water activities, and healthier eateries which are segmented based on certain regions of the island. Users can select the facility they are interested in, and the system will calculate the distance to nearby options, displaying one to three results based on proximity. |

# Appendix B: Analysis Models

## Use Case Model




# Appendix C: To Be Determined List

*<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>*

Source:
http://www.frontiernet.net/~kwiegers/process_assets/srs_template.doc